Design & Implementation of a Custom FPGA-Based Graphics Processing Unit (GPU)

"Find the Thief" — 2D Mystery Game with AI-Driven NPC Dialogue

---

Team Members:

- Tutika Sandeep
- Ayush
- Namburi Venkata Yeshwanth
- Ghaffar Salman
- Faisal Mohammed

Supervisor: Dr. Bhanu Prakash

Institution: School of Technology, Woxsen University

Hardware Platform:

- ESP32-S3-DevKitC-1 (Host CPU + AI Brain)
- Sipeed Tang Nano 20K (FPGA Graphics Engine)
- Cosmic Byte Blitz Gamepad (Player Input)
- HDMI Output at 640x480 @ 60Hz

Part 1: Project Specification

Custom FPGA GPU — FINAL Project Specification (v3)

Project Identity

Field	Details
Title	Design & Implementation of a Custom FPGA-Based Graphics Processing Unit (GPU)
Team	Tutika Sandeep, Ayush, Namburi Venkata Yeshwanth, Ghaffar Salman, Faisal Mohammed

Supervisor Dr. Bhanu Prakash
Institution School of Technology, Woxsen University
Game "Find the Thief" — 2D mystery village game with AI-driven NPC dialogue

---

Final Hardware BOM (Bill of Materials)

#	Component	Model	Role
1	Host CPU	**ESP32-S3-DevKitC-1** (dual USB-C)	Game logic, AI inference, gamepad input
2	FPGA GPU	**Sipeed Tang Nano 20K** (GW2AR-18)	Rendering engine, HDMI output
3	Gamepad	**Cosmic Byte Blitz**	Player input
4	Adapter	**USB-A to USB-C adapter**	Connects gamepad to ESP32-S3
5	Display	**Any HDMI monitor**	640×480 @ 60Hz output
6	SD Card	**microSD card** (any, 4GB+)	Stores AI model weights + game assets
7	Cables	Jumper wires (male-male)	SPI connection between ESP32-S3 ↔ Tang Nano 20K
8	Power	USB-C cables (×2)	One for each board

Engineering Decisions (Made by Me)

Decision 1: Color Depth → 8-bit Indexed (256-Color Palette)

Why this is the right choice for your project:

Factor	8-bit Indexed	16-bit Direct	Verdict
VRAM per frame	300 KB	600 KB	8-bit wins — half the memory
Double buffer	600 KB	1.2 MB	8-bit uses 7.5% of SDRAM vs 15%
FPGA complexity	Simple palette LUT in BRAM	Direct RGB mapping	8-bit simpler to implement
Art style match	Perfect for 2D pixel-art village	Overkill for this aesthetic	8-bit matches the game
Palette tricks	Can do fade effects by changing palette	No palette tricks possible	8-bit more versatile

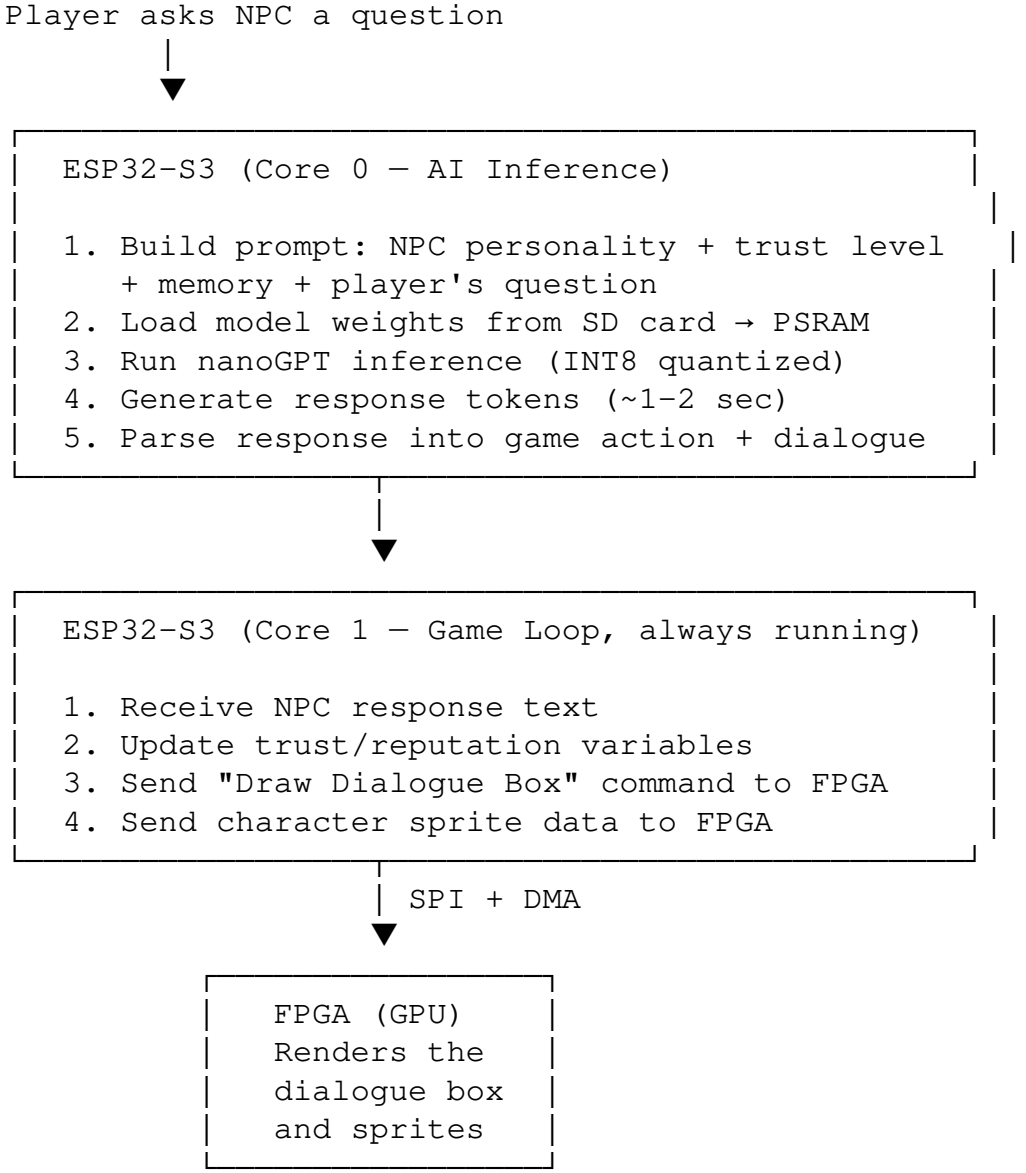
How it works: Each pixel stores a number 0–255. That number is an index into a 256-entry **Color Lookup Table (CLUT)** stored in BRAM. Each entry holds a full 18-bit or 24-bit RGB color. This gives you 256 simultaneous colors on screen from a total palette of 16 million.

Decision 2: AI Model Execution → Runs on ESP32-S3 (NOT the FPGA)

Why the ESP32-S3, not the FPGA:

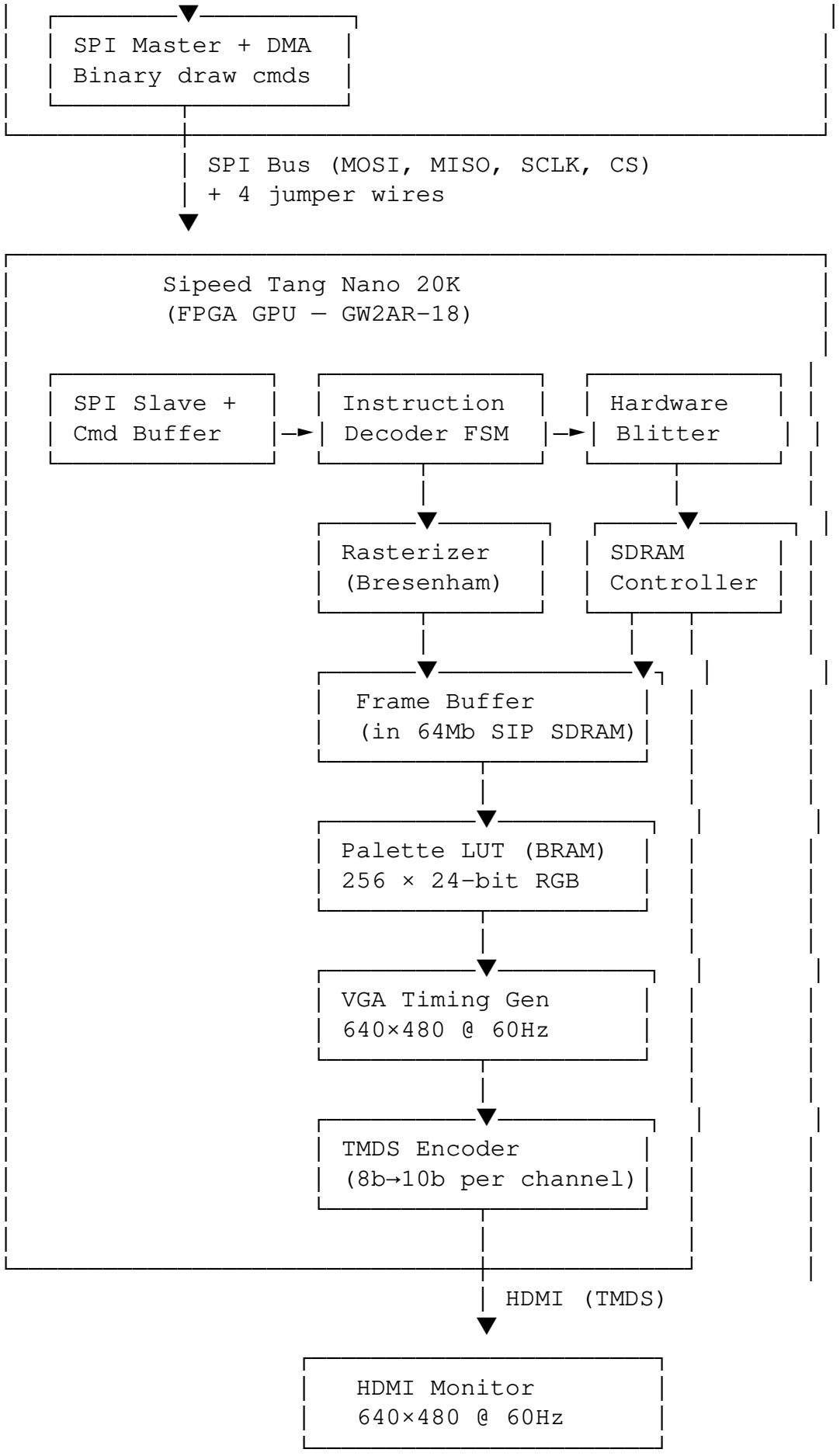
Factor	ESP32-S3	FPGA (Tang Nano 20K)
Architecture	Sequential CPU — perfect for transformer's layer-by-layer inference	Parallel logic gates — designed for pixel-pushing
Memory access	Can stream weights from PSRAM/SD card sequentially	SDRAM is shared with framebuffer — contention risk
Proven feasibility	People have run 15M+ param models on ESP32-S3	No known transformer inference on 20K LUT FPGA
Development effort	Use ESP-DL or TFLite Micro frameworks	Would require custom Verilog matrix multiply engine
Latency tolerance	NPC dialogue has 1–2 seconds of acceptable wait time	GPU needs sub-microsecond response for 60 FPS

The AI pipeline:



nanoGPT specs for your project:

Parameter	Value
Architecture	Transformer (decoder-only, GPT-2 style)
Parameters	~5 million


SPI Master + DMA
Binary draw cmds
SPI Bus (MOSI, MISO, SCLK, CS)
+ 4 jumper wires
Sipeed Tang Nano 20K
(FPGA GPU — GW2AR-18)
SPI Slave +
Cmd Buffer
Instruction
Decoder FSM
Hardware
Blitter
Rasterizer
(Bresenham)
SDRAM
Controller
Frame Buffer
(in 64Mb SIP SDRAM)
Palette LUT (BRAM)
256 × 24-bit RGB
VGA Timing Gen
640×480 @ 60Hz
TMDS Encoder
(8b→10b per channel)
HDMI (TMDS)
HDMI Monitor
640×480 @ 60Hz

Memory Map

ESP32-S3 Memory

Memory	Size	Contents
Internal SRAM	512 KB	Game variables, NPC states, trust arrays, active buffers
PSRAM	8 MB	nanoGPT weights (loaded from SD), inference workspace
microSD Card	4+ GB	AI model file (~5MB), sprite sheet PNGs, dialogue corpus, map data

FPGA (Tang Nano 20K) Memory

Memory	Size	Contents
BRAM (828 Kbit)	~103 KB	Color palette LUT (768 bytes), sprite tile cache, line buffers, SPI receive buffer
SIP SDRAM (64 Mbit)	8 MB	Frame buffer A (300 KB) + Frame buffer B (300 KB) + sprite sheet data + village tilemap

Frame Buffer Budget

Item	Size	Running Total
Frame Buffer A (640×480×8bit)	300 KB	300 KB
Frame Buffer B (double buffer)	300 KB	600 KB
Sprite sheet storage	~2 MB	2.6 MB
Village tilemap	~500 KB	3.1 MB
Free headroom	**4.9 MB**	**8 MB total**

FPGA Modules (All in Verilog)

#	Module	LUT Estimate	Description
1	`vga_timing.v`	~200	H/V counters, HSYNC, VSYNC, video_active
2	`tmds_encoder.v`	~600	3× TMDS channels (R, G, B) + clock, 8b→10b
3	`spi_slave.v`	~300	Receives binary commands from ESP32-S3
4	`cmd_decoder.v`	~500	FSM parsing draw commands (sprite, line, clear, text)
5	`blitter.v`	~800	Block transfer engine for sprite rendering
6	`rasterizer.v`	~400	Bresenham's line + rectangle fill
7	`sdram_ctrl.v`	~1500	SDRAM init, refresh, read/write state machine
8	`palette_lut.v`	~100	256-entry CLUT, 8-bit index → 24-bit RGB
9	`gpu_top.v`	—	Top-level wiring everything together
	TOTAL	**~4,400**	**Uses ~21% of 20,736 LUTs** plenty of room

---

Communication Protocol (ESP32-S3 → FPGA)

SPI Command Format

```
┌──────────┬──────────┬──────────────────────┐
│ CMD (8b) │ LEN (8b) │ PAYLOAD (N bytes)    │
└──────────┴──────────┴──────────────────────┘
```

Command	Code	Payload	Description
NOP	`0x00`	—	No operation
CLEAR	`0x01`	1 byte (color index)	Fill entire screen with palette color
DRAW_SPRITE	`0x02`	x(16b) + y(16b) + sprite_id(16b) + flags(8b)	Blit a sprite from SDRAM to framebuffer
DRAW_LINE	`0x03`	x0(16b) + y0(16b) + x1(16b) + y1(16b) + color(8b)	Bresenham line
DRAW_RECT	`0x04`	x(16b) + y(16b) + w(16b) + h(16b) + color(8b)	Filled rectangle
SET_PALETTE	`0x05`	index(8b) + R(8b) + G(8b) + B(8b)	Update one palette entry
SWAP_BUFFER	`0x06`	—	Swap front/back framebuffer (vsync)
LOAD_SPRITE	`0x07`	addr(24b) + data(Nb)	Upload sprite data to SDRAM

Game Data Flow (One Frame)

```
1. ESP32-S3 Core 1 polls gamepad (USB HID) → player presses D-pad right
2. Game logic moves player sprite X position +2 pixels
3. If player is near NPC and presses A → trigger dialogue
4. Core 0 runs nanoGPT inference → generates NPC response (~1-2 sec)
5. Core 1 sends SPI commands to FPGA:
   a. CLEAR(sky_blue)           — clear back buffer
   b. DRAW_SPRITE(bg_tiles...)  — draw village background
   c. DRAW_SPRITE(npc_id, x, y) — draw NPC sprite
   d. DRAW_SPRITE(player, x, y) — draw player sprite
   e. DRAW_RECT(dialogue_box)   — draw dialogue box
   f. [text rendering commands]  — draw NPC response text
   g. SWAP_BUFFER               — flip to display
6. FPGA reads back buffer from SDRAM, converts through palette LUT
7. TMDS encoder serializes RGB → HDMI output to monitor
8. Repeat at 60 FPS
```

Software Stack

Layer	Tool	Purpose
FPGA Synthesis	**Gowin EDA**	Compile Verilog → bitstream for GW2AR-18
FPGA Simulation	**Icarus Verilog** + **GTKWave**	Pre-synthesis timing verification
ESP32-S3 Firmware	**Arduino IDE / PlatformIO**	Game logic + AI inference in C

AI Training	**nanoGPT (Python, PyTorch)**	Train 5M param model on dialogue corpus
AI Quantization	**ESP-DL / TFLite Micro**	Convert to INT8 for embedded deployment
Gamepad Library	**EspUsbHost**	USB HID gamepad reading on ESP32-S3

Part 2: nanoGPT AI System

nanoGPT AI System — Complete Plan

> "**Find the Thief" NPC Dialogue Engine** Trained by us, runs on the ESP32-S3, makes NPCs feel alive.

1. What We're Building

A **tiny transformer language model** (~5M parameters) that generates contextual NPC dialogue responses based on:

- Who the NPC is (personality)
- How much they trust the player (trust level 0–100)
- What they've seen/experienced (memory)
- What the player just said/did (current input)

This replaces hundreds of hand-written dialogue trees with a single model that **generates** responses.

2. Model Architecture

We're using **nanoGPT** by Andrej Karpathy — a minimal, clean GPT-2 implementation in PyTorch.

Configuration for ~5M Parameters

```
# config/train_thief.py — nanoGPT training config

# Model architecture
n_layer = 4          # 4 transformer blocks (lightweight but capable)
n_head = 6           # 6 attention heads
n_embd = 192         # 192-dimensional embeddings
block_size = 256     # 256 character context window
dropout = 0.1        # 10% dropout for regularization

# Tokenization
```

```
# Character-level (NOT BPE) — simpler, smaller vocab, perfect for embed
# Vocab size ≈ 80-90 characters (letters, digits, punctuation, special

# Training
batch_size = 32
learning_rate = 3e-4
max_iters = 10000    # ~30 min on a laptop GPU
eval_interval = 500
eval_iters = 200
device = 'cuda'      # or 'cpu' if no GPU (slower but works)
```

Parameter Breakdown

Component	Formula	Parameters
Token Embedding	vocab_size × n_embd	90 × 192 = **17,280**
Position Embedding	block_size × n_embd	256 × 192 = **49,152**
Per Transformer Block	$12 \times n_embd^2 + 13 \times n_embd$	12 × 36,864 + 2,496 = **444,864**
× 4 blocks		**1,779,456**
LayerNorm (final)	2 × n_embd	**384**
Output Head	n_embd × vocab_size	192 × 90 = **17,280**
TOTAL		**~1.86M**

Final Recommended Config

```
# FINAL — ~4.8M parameters
n_layer = 6
n_head = 8
n_embd = 256
block_size = 256     # 256 chars context (enough for prompt + response)
vocab_size = 90      # character-level
```

Metric	Value
Parameters	~4.8 million
FP32 size	~19 MB
INT8 quantized size	**~4.8 MB** ← fits in ESP32-S3 PSRAM

---

3. Training Data — How I Help You

This is where I come in. You'll ask me to generate the training corpus, and I'll produce hundreds of structured dialogue samples.

Training Data Format

Each training example follows this exact structure:

```
<|npc|>SHOPKEEPER<|trust|>HIGH<|memory|>player helped organize shop<|mo
```

The Prompt Template

```
<|npc|>{NPC_NAME}<|trust|>{LOW/MEDIUM/HIGH}<|memory|>{what this NPC rem
```

The model learns to complete everything after `<|response|>` and stop at `<|end|>`.

NPC Profiles (What I'll Bake Into the Training Data)

Shopkeeper (Ravi)

- **Personality:** Friendly but calculative, knows everyone's business, gossips
- **Secret:** He's the actual thief (or red herring — your choice)
- **Speech style:** Warm, uses phrases like "my friend," deflects accusations smoothly
- **Low trust:** Gives generic helpful advice, hides information
- **High trust:** Reveals secrets about other NPCs, may slip up about the theft

Guard (Karan)

- **Personality:** Stern, proud, takes his job seriously, gets angry if accused
- **Secret:** Was asleep on duty the night of the theft (embarrassed about it)
- **Speech style:** Formal, short sentences, military-like
- **Low trust:** Dismissive, "That's none of your concern, citizen"
- **High trust:** Admits he fell asleep, shares patrol route details

Old Man (Thakur Ji)

- **Personality:** Wise, cryptic, speaks in riddles/metaphors, knows village history
- **Secret:** Saw something from his window but is afraid to speak
- **Speech style:** Slow, philosophical, "In my day..." type stories
- **Low trust:** Tells irrelevant old stories, avoids the topic
- **High trust:** Reveals what he saw — a shadow carrying something heavy

Kid (Chintu)

- **Personality:** Curious, energetic, sneaks around at night, sees everything
- **Secret:** Saw the shopkeeper carrying a bag near the vault
- **Speech style:** Excited, uses "bhaiya" / informal language, easily distracted
- **Low trust:** Talks about games and random stuff, doesn't help
- **High trust:** Blurts out what he saw — crucial clue

Farmer (Baldev)

- **Personality:** Hardworking, quiet, suspicious of outsiders, honest
- **Secret:** His tools were stolen too (possibly used in the heist)
- **Speech style:** Simple, direct, few words
- **Low trust:** "I have work to do. Leave me alone."
- **High trust:** Mentions his missing tools, connects dots

Training Data Volume Target

Category	Count
NPCs	5
Trust levels	3 (LOW, MEDIUM, HIGH)
Scenario types	~10 (greeting, accusation, asking for help, giving clue, refusing, etc.)
Variations per scenario	3–5
Total training samples	**~500–750 dialogues**
Characters per sample	~200–400
Total training text	**~150,000–250,000 characters**

How We Generate This Together

Step 1: I generate the base corpus. You tell me:

```
"Generate 50 training dialogues for the Shopkeeper at all trust levels"
```

And I produce 50 structured samples like:

```
<|npc|>SHOPKEEPER<|trust|>LOW<|memory|>none<|mood|>neutral<|input|>Hell

<|npc|>SHOPKEEPER<|trust|>LOW<|memory|>none<|mood|>nervous<|input|>Wher

<|npc|>SHOPKEEPER<|trust|>HIGH<|memory|>player helped organize shop,pla
```

Step 2: You review/edit them for tone and accuracy.

Step 3: We repeat for all 5 NPCs.

Step 4: I also generate "edge case" samples — wrong accusations, repeated questions, contradictions.

---

4. Training Pipeline (On Your PC)

Prerequisites

```
# Install Python (if not already installed)
# Download from python.org — make sure to check "Add to PATH"

# Clone nanoGPT
cd "D:\CP GPU"
git clone https://github.com/karpathy/nanoGPT.git
cd nanoGPT

# Install dependencies
pip install torch numpy transformers datasets tiktoken tqdm
```

Step-by-Step Training

```
D:\CP GPU\nanoGPT\
├── data/
│   └── find_the_thief/
│       ├── input.txt          ← All 500+ dialogues concatenated
│       └── prepare.py         ← Character-level tokenizer script
├── config/
│   └── train_thief.py         ← Our custom training config
├── train.py                   ← nanoGPT trainer (already in repo)
└── sample.py                  ← Text generation script (already in rep
```

Step 1: Prepare the data

```
# After I generate all dialogues and you save them to input.txt:
cd "D:\CP GPU\nanoGPT"
python data/find_the_thief/prepare.py
# This creates train.bin and val.bin
```

Step 2: Train

```
python train.py config/train_thief.py
# ~30 minutes on a laptop GPU (GTX 1650 or better)
# ~2-4 hours on CPU (still doable)
# Outputs: out-thief/ckpt.pt (the trained model)
```

Step 3: Test generation

```
python sample.py --out_dir=out-thief --start="<|npc|>GUARD<|trust|>LOW<
# Should generate a Guard-like angry denial response
```

---

5. Deployment Pipeline (PC → ESP32-S3)

```
graph LR
    A["PyTorch Model<br/>ckpt.pt (~19 MB)"] -->|"torch.onnx.export()"|
    B -->|"ESP-PPQ Quantizer"| C["INT8 Model<br/>model_int8.bin (~4.8 M
    C -->|"Copy to SD card"| D["microSD Card"]
    D -->|"Loaded at boot"| E["ESP32-S3 PSRAM<br/>(Runtime Inference)"]
```

Step-by-Step Conversion

```
# export_model.py — Run on your PC after training

import torch
from model import GPT, GPTConfig

# 1. Load trained model
config = GPTConfig(
    n_layer=6, n_head=8, n_embd=256,
    block_size=256, vocab_size=90
)
model = GPT(config)
```

```
checkpoint = torch.load('out-thief/ckpt.pt', map_location='cpu')
model.load_state_dict(checkpoint['model'])
model.eval()

# 2. Export to ONNX
dummy_input = torch.randint(0, 90, (1, 256))  # batch=1, seq=256
torch.onnx.export(
    model, dummy_input, "model.onnx",
    input_names=['input_ids'],
    output_names=['logits'],
    opset_version=13
)
print("Exported to ONNX!")

# 3. Quantize with ESP-PPQ (Espressif's quantization tool)
# pip install esp-ppq
# python -m esp_ppq quantize --input model.onnx --output model_int8.bin
```

SD Card Structure

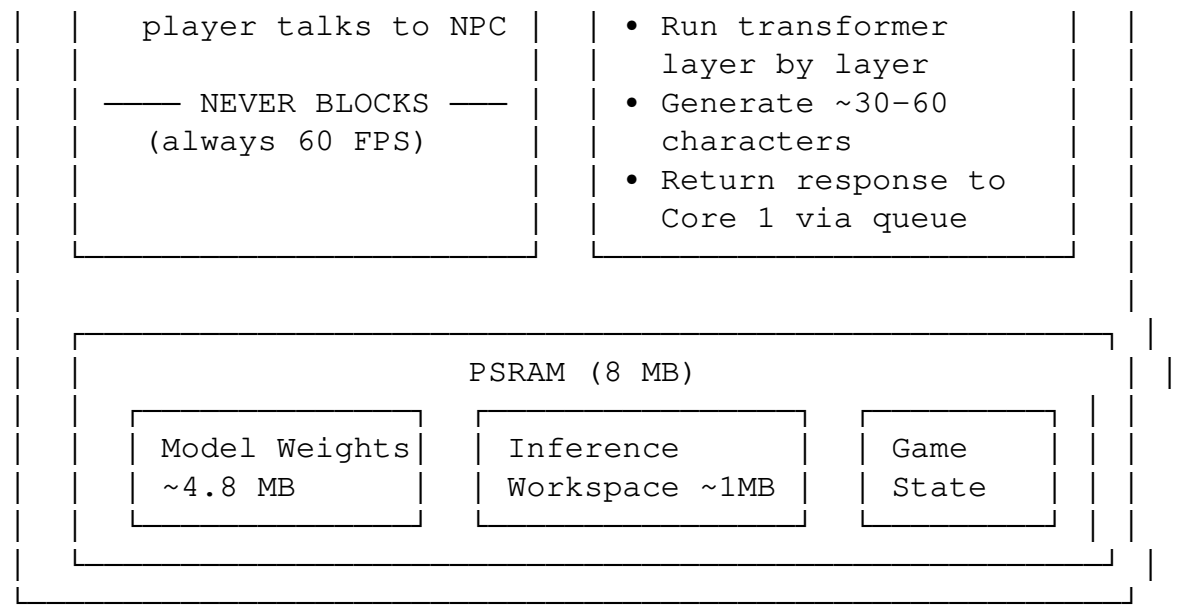
```
microSD Card (FAT32)
├── model/
│   ├── model_int8.bin        ← 4.8 MB quantized weights
│   ├── vocab.bin             ← Character-to-integer mapping
│   └── config.bin            ← Model config (layers, dims, etc.)
├── sprites/
│   ├── shopkeeper.bin        ← Sprite data for FPGA
│   ├── guard.bin
│   └── ...
└── maps/
    └── village.bin           ← Tilemap data
```

---

6. Runtime on ESP32-S3

Dual-Core Architecture

```
┌──────────────────────────────────────────────────────────┐
│                    ESP32-S3 Runtime                      │
│                                                          │
│  ┌──────────────────────────┐  ┌──────────────────────┐  │
│  │ CORE 1 (Game Loop)       │  │ CORE 0 (AI Inference)│  │
│  │ Priority: REALTIME       │  │ Priority: NORMAL     │  │
│  │                          │  │                      │  │
│  │ • Poll gamepad (USB)     │  │ • Idle until triggered  │
│  │ • Update game state      │  │ • Build prompt string│  │
│  │ • Collision detection    │  │ • Tokenize (char→int)│  │
│  │ • Send draw cmds SPI     │  │ • Load weights from  │  │
│  │ • Manage NPC states      │  │   PSRAM (pre-loaded  │  │
│  │ • Trigger AI when        │  │   from SD at boot)   │  │
```



Prompt Construction at Runtime

When the player talks to an NPC, Core 1 sends this to Core 0:

```
// game_ai.h — Prompt builder

typedef struct {
    char npc_name[16];      // "SHOPKEEPER"
    uint8_t trust_level;    // 0–100 → mapped to LOW/MEDIUM/HIGH
    char memory[128];       // "player helped organize shop"
    char mood[16];          // "friendly"
    char player_input[128]; // "Did you see anything?"
} AIRequest;

// Core 1 builds and sends:
AIRequest req = {
    .npc_name = "KID",
    .trust_level = 75,       // HIGH
    .memory = "player gave him candy,player played catch",
    .mood = "excited",
    .player_input = "Did you see anything the night the gold disappeare
};
xQueueSend(ai_queue, &req, portMAX_DELAY);

// Core 0 constructs the prompt string:
// "<|npc|>KID<|trust|>HIGH<|memory|>player gave him candy,player playe
// Then runs inference character-by-character until <|end|> token
```

Expected Performance

Metric	Value
Model load time (SD → PSRAM)	~2 seconds at boot
Prompt tokenization	< 1 ms

Inference per token	~30–50 ms
Typical response length	30–60 characters
Total response time	**~1–3 seconds**
Player perception	Shows "..." animation while thinking → feels natural

---

7. Fallback System (Safety Net)

If the AI model is too slow or produces garbage, we have a backup:

```
// fallback_dialogue.h — Pre-written responses indexed by NPC + trust +

const char* fallback_responses[5][3][10] = {
    // [NPC_SHOPKEEPER][TRUST_LOW][SCENARIO_GREETING]
    { { "Welcome to my shop! Looking to buy something?",
        "Ah, a visitor! Browse around, take your time.",
        ... } },
    // [NPC_SHOPKEEPER][TRUST_HIGH][SCENARIO_ACCUSATION]
    { { "After all I've done for you? I'm hurt, truly hurt.",
        ... } },
    ...
};

// Use fallback if:
// 1. AI inference exceeds 5 second timeout
// 2. Generated text is too short (<10 chars) or too long (>200 chars)
// 3. Generated text doesn't end with <|end|> token
```

---

8. Timeline for the AI Component

Phase	Task	Time	Who Does It
Phase 1	I generate 500+ dialogue samples	1 session	You ask me, I produce
Phase 2	You review/edit the dialogues	1–2 days	You
Phase 3	Install nanoGPT + train model	1 day	You (I guide step-by-step)
Phase 4	Test generation on PC	1 day	You (I help debug)
Phase 5	Quantize + deploy to ESP32-S3	2–3 days	We do together
Phase 6	Integrate with game loop	2–3 days	We do together
Phase 7	Test + tune	Ongoing	You play, we iterate

Part 3: Game Development Plan

Game Building Plan — "Find the Thief"

1. Game Overview

A 2D top-down mystery game set in a pixel-art village. The player walks around, talks to 5 NPCs, gathers clues, and accuses the thief. NPC responses are generated by a nanoGPT AI model running on the ESP32-S3.

---

2. Art Pipeline — Creating Characters & World

Tools Required

Tool	Purpose	Cost
Aseprite	Pixel art sprites + animations	$20 (Steam) or free via source compile
Tiled Map Editor	Village tilemap design	Free (mapeditor.org)
Python (Pillow)	Convert PNG sprites → raw binary for FPGA	Free

Sprite Specifications

Asset	Size	Frames	Color Depth
Player character	16×32 px	4 per direction (walk) + 1 idle = 17 frames	8-bit indexed
Each NPC (×5)	16×32 px	2 idle animation frames each	8-bit indexed
Village tiles	16×16 px	~40 unique tiles (grass, path, wall, water, roof, door, fence, tree)	8-bit indexed
Dialogue box	280×80 px	1 static frame	8-bit indexed
Font glyphs	8×8 px	96 ASCII characters (A-Z, a-z, 0-9, punctuation)	1-bit (monochrome)
UI elements	Various	Cursor, trust meter, accuse button	8-bit indexed

Character Design Guide

Each NPC needs a visually distinct silhouette even at 16×32:

NPC	Visual Identity	Key Color
Shopkeeper (Ravi)	Apron, round body, mustache	Warm brown/orange
Guard (Karan)	Helmet, spear, upright posture	Steel blue/grey
Old Man (Thakur Ji)	Walking stick, hunched, long beard	Muted green/white

Kid (Chintu)	Small height (16×24), cap, energetic pose	Bright yellow/red
Farmer (Baldev)	Straw hat, broad shoulders, hoe on back	Earth brown/green

Art Creation Workflow

```
Step 1: Create a 256-color palette in Aseprite (shared across all sprit
        → Export as palette.pal

Step 2: Draw each character on separate Aseprite files
        → Use Onion Skinning for animation frames
        → Export each as a sprite sheet PNG (horizontal strip)

Step 3: Draw all 16×16 tiles in one Aseprite tileset file
        → Export as tileset.png

Step 4: Open Tiled Map Editor → import tileset.png
        → Paint the village map (40×30 tiles = 640×480 pixels)
        → Mark collision zones and NPC spawn points
        → Export as village.json

Step 5: Run conversion script (Python) to convert everything to binary:
        → sprite PNGs → .bin files (raw palette indices)
        → village.json → village.bin (tile index array)
        → Copy all .bin files to microSD card
```

Sprite-to-Binary Conversion Script

```
# tools/convert_sprites.py
from PIL import Image
import struct

def png_to_bin(input_png, output_bin, palette):
    """Convert indexed-color PNG to raw binary for FPGA."""
    img = Image.open(input_png).convert('P')  # Palette mode
    pixels = list(img.getdata())
    with open(output_bin, 'wb') as f:
        f.write(struct.pack('<HH', img.width, img.height))  # Header
        f.write(bytes(pixels))  # Raw palette indices
    print(f"Converted {input_png} → {output_bin} ({len(pixels)} bytes)"

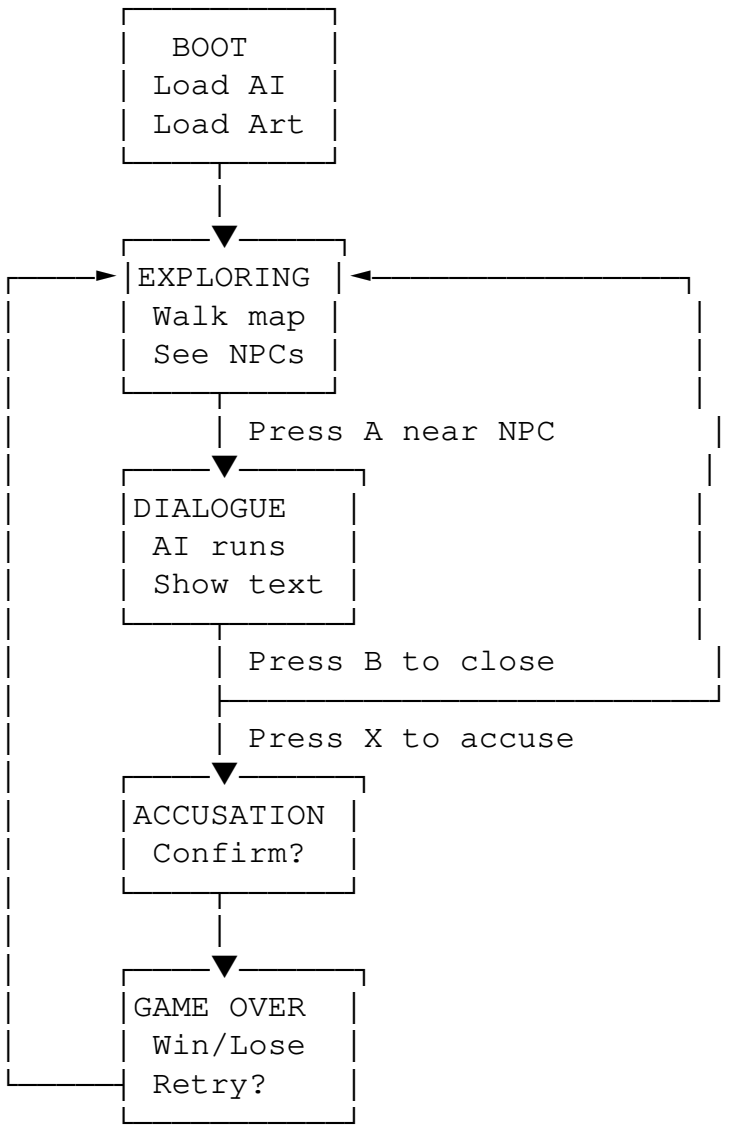
def convert_tilemap(json_path, output_bin):
    """Convert Tiled JSON export to binary tilemap."""
    import json
    with open(json_path) as f:
        data = json.load(f)
    layer = data['layers'][0]  # Ground layer
    tiles = layer['data']
    w, h = layer['width'], layer['height']
    with open(output_bin, 'wb') as f:
```

```
        f.write(struct.pack('<HH', w, h))
        for tile_id in tiles:
            f.write(struct.pack('<B', tile_id & 0xFF))
    print(f"Converted tilemap: {w}×{h} tiles → {output_bin}")
```

---

3. Game State Machine

The game runs on ESP32-S3 Core 1 as a finite state machine:



State Descriptions

State	What Happens	Inputs	FPGA Rendering
BOOT	Load AI model from SD to PSRAM, load sprite data to FPGA SDRAM	None (automatic)	Splash screen

EXPLORING	Player walks around village with D-pad, camera follows	D-pad: move, A: interact	Village tilemap + all NPC sprites + player sprite
DIALOGUE	AI generates NPC response, text appears character-by-character	A: next line, B: close	Village + dialogue box overlay + NPC portrait
ACCUSATION	Player selects an NPC to accuse as the thief	D-pad: select, A: confirm, B: cancel	NPC lineup + selection cursor
GAME_OVER	Win (correct accusation) or Lose (wrong accusation)	A: retry	Result screen with story summary

---

4. ESP32-S3 Game Engine Code Structure

```
D:\CP GPU\firmware\
├── main/
│   ├── main.c              ← Entry point, FreeRTOS task creation
│   ├── game_loop.c         ← Core 1: state machine, 60 FPS tick
│   ├── game_states.h       ← State enum + transition logic
│   ├── player.c            ← Player movement, collision, position
│   ├── npc.c               ← NPC data: position, trust, memory, mood
│   ├── npc.h               ← NPC struct definitions
│   ├── dialogue.c          ← Dialogue box rendering commands
│   ├── input.c             ← USB HID gamepad polling (EspUsbHost)
│   ├── gpu_commands.c      ← SPI command builder (DRAW_SPRITE, etc.)
│   ├── gpu_commands.h      ← Command codes and packet structs
│   ├── ai_inference.c      ← Core 0: nanoGPT INT8 inference engine
│   ├── ai_inference.h      ← AIRequest struct, queue interface
│   ├── asset_loader.c      ← SD card → PSRAM/FPGA asset loading
│   └── map.c               ← Tilemap loader, collision map, camera
├── components/
│   ├── esp_usb_host/       ← EspUsbHost library
│   └── esp_dl/             ← ESP-DL inference framework
└── sdkconfig               ← ESP-IDF configuration
```

Core Game Loop (Pseudocode)

```
// game_loop.c — Runs on Core 1 at 60 FPS

void game_loop_task(void *param) {
    GameState state = STATE_BOOT;
    Player player = {.x = 320, .y = 240, .sprite_id = 0};
    NPC npcs[5];
    init_npcs(npcs);

    while (1) {
        uint32_t frame_start = millis();
        GamepadInput input = poll_gamepad();
```

```
        switch (state) {
            case STATE_BOOT:
                gpu_cmd_clear(COLOR_BLACK);
                gpu_cmd_draw_sprite(SPLASH_SCREEN, 0, 0);
                gpu_cmd_swap_buffer();
                load_ai_model();        // SD → PSRAM
                load_sprites_to_fpga(); // SD → SPI → FPGA SDRAM
                state = STATE_EXPLORING;
                break;

            case STATE_EXPLORING:
                update_player(&player, input);
                check_npc_proximity(&player, npcs, &state);

                // Render frame
                gpu_cmd_clear(COLOR_SKY);
                draw_tilemap(player.camera_x, player.camera_y);
                for (int i = 0; i < 5; i++)
                    gpu_cmd_draw_sprite(npcs[i].sprite_id, npcs[i].x, n
                gpu_cmd_draw_sprite(player.sprite_id, player.screen_x,
                gpu_cmd_swap_buffer();
                break;

            case STATE_DIALOGUE:
                if (ai_response_ready()) {
                    char *response = get_ai_response();
                    draw_dialogue_box(response, current_npc);
                }
                if (input.b_pressed) state = STATE_EXPLORING;
                if (input.x_pressed) state = STATE_ACCUSATION;
                gpu_cmd_swap_buffer();
                break;

            case STATE_ACCUSATION:
                draw_accusation_screen(npcs);
                if (input.a_pressed) {
                    if (selected_npc == THE_THIEF)
                        state = STATE_WIN;
                    else
                        state = STATE_LOSE;
                }
                break;
        }

        // Frame rate limiter: 16.67ms per frame = 60 FPS
        uint32_t elapsed = millis() - frame_start;
        if (elapsed < 16) vTaskDelay(pdMS_TO_TICKS(16 - elapsed));
    }
}
```

5. NPC Interaction System

Trust Mechanics

Action	Trust Change
Help an NPC (do their side task)	+20
Give an item they want	+15
Talk politely	+5
Wrongly accuse them	-30
Ask rude questions	-10
Ignore them for a long time	-5

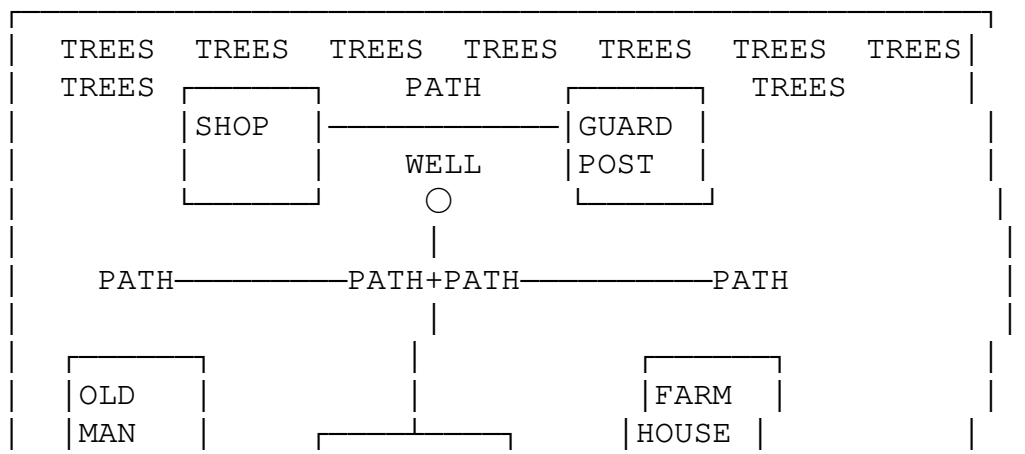
Trust Level Thresholds

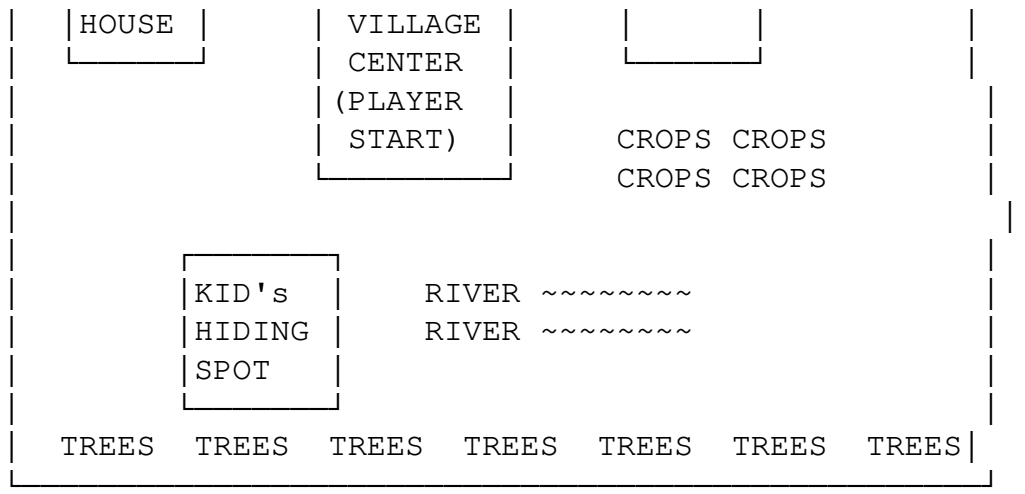
Range	Label	NPC Behavior
0–30	LOW	Dismissive, unhelpful, generic responses
31–65	MEDIUM	Somewhat helpful, gives vague hints
66–100	HIGH	Shares secrets, reveals clues, trusts player

Side Tasks (Trust Builders)

NPC	Side Task	Reward
Shopkeeper	Help organize crates (mini-game or auto)	+20 trust with Shopkeeper
Guard	Find his missing badge	+20 trust with Guard
Old Man	Bring him his medicine from the shop	+20 trust with Old Man
Kid	Play catch / find his lost toy	+20 trust with Kid
Farmer	Water his crops (carry bucket from well)	+20 trust with Farmer

6. Village Map Layout





Map: 40×30 tiles (16px each) = 640×480 pixels (exactly fills the screen — no scrolling needed for MVP).

---

7. Development Phases — Start to Finish

Phase 1: Foundation (Week 1–2)

Task	Details	Output
Install all tools	Aseprite, Tiled, Gowin EDA, Icarus Verilog, Arduino IDE	Working dev environment
Create color palette	256-color palette shared by all art assets	palette.pal
Draw player sprite	16×32, idle + 4-direction walk animation	player.aseprite → player.png
Draw 5 NPC sprites	16×32 each, idle animation (2 frames)	npc_*.png
Draw village tileset	16×16 tiles (~40 tiles)	tileset.png
Design village map	In Tiled, place tiles, mark collision, NPC spawns	village.json

Phase 2: FPGA GPU Core (Week 2–4)

Task	Details	Output
Simulate VGA timing	Verify HSYNC/VSYNC/video_active in GTKWave	Simulation screenshots
Implement SDRAM controller	Use nand2mario's open-source controller as reference	sdram_ctrl.v
Implement TMDS encoder	3-channel 8b→10b encoding for HDMI	tmds_encoder.v
Implement palette LUT	256-entry BRAM color lookup	palette_lut.v

Presentation prep	GTKWave screenshots, live demo, documentation	Presentation materials

8. Asset File Sizes (Estimated)

Asset	Calculation	Size
Player sprite (17 frames × 16×32)	17 × 512 bytes	~8.5 KB
5 NPC sprites (2 frames × 16×32 each)	5 × 2 × 512	~5 KB
Village tileset (40 tiles × 16×16)	40 × 256	~10 KB
Village tilemap (40×30 indices)	1,200 bytes	~1.2 KB
Font glyphs (96 chars × 8×8 / 8)	96 × 8	~768 bytes
Dialogue box frame	280×80	~22 KB
Color palette (256 × 3 bytes RGB)	768 bytes	~768 bytes
Total art assets		**~48 KB**
AI model (INT8)		~4.8 MB
Grand total on SD card		**~5 MB**

9. Testing Checklist

#	Test	Pass Criteria
1	FPGA shows solid color via HDMI	Monitor displays uniform color
2	Player sprite renders on screen	Character visible at center
3	D-pad moves player	Smooth 4-direction movement
4	Tilemap renders correctly	Village appears as designed in Tiled
5	NPC sprites visible at correct positions	All 5 NPCs in their map locations
6	Pressing A near NPC triggers dialogue	Dialogue box appears
7	AI generates contextual response	Response matches NPC personality + trust
8	Trust changes with player actions	Helping NPC increases trust, accusations decrease
9	Accusation flow works	Correct accusation = win, wrong = lose
10	Stable 60 FPS during exploration	No visible stuttering or tearing